

NAME: MAYANK MEHTA

USN: 1BF24CS171

HILL CLIMBING ALGORITHM [4 QUEENS]:

CODE:

```
def calculate_cost(state):
    """
    Calculates heuristic cost h(s): the number of pairs of queens
    attacking each other
    directly or indirectly (row or diagonal).
    """
    attacking_pairs = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            # Check for same row attack
            if state[i] == state[j]:
                attacking_pairs += 1
            # Check for diagonal attack
            elif abs(state[i] - state[j]) == abs(i - j):
                attacking_pairs += 1
    return attacking_pairs

def get_successors(state):
    """
    Generates all successors by moving a single queen to another row in
    its column
    (4 * 3 = 12 successors for a 4x4 board).
    """
    successors = []
    n = len(state)
    for col in range(n):
        for row in range(n):
            if row != state[col]:
                next_state = list(state)
                next_state[col] = row
                successors.append(tuple(next_state))
    return successors

def hill_climbing(initial_state):
```

```

"""
    Steepest-descent Hill Climbing algorithm minimizing attacking pairs
    h(s).
"""
current = tuple(initial_state)
current_cost = calculate_cost(current)
print(f"Initial State: {current}, Heuristic Cost (Attacking Pairs h):
{current_cost}")

step = 0
while True:
    step += 1
    successors = get_successors(current)
    best_successor = None
    best_cost = current_cost

    # Evaluate all successors to find the one with the lowest cost
    for succ in successors:
        cost = calculate_cost(succ)
        if cost < best_cost:
            best_cost = cost
            best_successor = succ

    # If no neighbor improves the cost, terminate (reached
    local/global minimum)
    if best_successor is None:
        print(f"Terminated at step {step}: Local/Global optimum
reached.")
        break

    print(f"Step {step}: Move from {current} (h={current_cost}) ->
{best_successor} (h={best_cost})")
    current = best_successor
    current_cost = best_cost

    return current, current_cost

# Execution with the given input state: c0=1, c1=1, c2=0, c3=2
initial_state = (1, 1, 0, 2)
final_state, final_cost = hill_climbing(initial_state)

```

```
print(f"\nFinal Answer State: {final_state}")  
print(f"Final Cost h(s): {final_cost}")
```

OUTPUT:

```
• Initial State: (1, 1, 0, 2), Heuristic Cost (Attacking Pairs h): 2  
  Step 1: Move from (1, 1, 0, 2) (h=2) -> (1, 3, 0, 2) (h=0)  
  Terminated at step 2: Local/Global optimum reached.  
  
  Final Answer State: (1, 3, 0, 2)  
  Final Cost h(s): 0
```